

Santos Pegasus Soluciones

Guia Oficial de Ingenieria Back-end

Stack tecnologico

El equipo de backend utiliza el siguiente stack:

- Lenguaje: Python 3.11+
- Framework: FastAPI (APIs REST) con Pydantic v2 para validacion
- Base de datos principal: PostgreSQL 15 (ORM: SQLAlchemy 2.0 con Alembic para migraciones)
- Cache: Redis 7 (sesiones, rate limiting, resultados costosos)
- Cola de mensajes: RabbitMQ (eventos entre microservicios)
- Testing: pytest + pytest-asyncio, cobertura minima del 80%
- Contenedores: Docker + docker-compose para desarrollo local

Estructura de proyecto

Todos los microservicios siguen la misma estructura de directorios:

```
src/  
  api/      # Routers FastAPI  
  domain/   # Modelos de dominio y logica de negocio  
  infrastructure/ # Repositorios, clientes externos, ORM  
  core/     # Configuracion, dependencias compartidas  
tests/  
  unit/  
  integration/  
alembic/    # Migraciones de base de datos  
Dockerfile  
pyproject.toml
```

Estandares de API REST

Todas las APIs deben seguir estos estandares:

- Versionado en la URL: /api/v1/recurso
- Respuestas de error con formato: {error: codigo, message: descripcion, details: [...]}
- Paginacion mediante query params: ?page=1&size=20
- Autenticacion: JWT Bearer token con expiracion de 24 horas
- Refresh token: expiracion de 30 dias, rotation habilitada

- Rate limiting: 100 req/min por usuario autenticado, 20 req/min anonimo
- Todos los endpoints deben tener OpenAPI docs completos (descripcion, responses, ejemplos)

Autenticacion y seguridad

El servicio de autenticacion es auth-service. Todos los demas servicios validan tokens mediante el middleware compartido en el paquete interno santospegasus-commons.

Reglas de seguridad:

- Nunca almacenar passwords en texto plano. Usar bcrypt con cost factor 12.
- Variables de entorno sensibles siempre en el vault de secretos (HashiCorp Vault).
- SQL: usar siempre ORM o queries parametrizadas. Prohibido SQL dinamico con f-strings.
- CORS: configurado explicitamente, nunca usar allow_origins=["*"] en produccion.

Testing y calidad

Requisitos de calidad para cada PR:

- Cobertura de tests $\geq 80\%$ en el modulo modificado.
- Todos los tests unitarios deben pasar en menos de 30 segundos.
- Los tests de integracion usan una base de datos PostgreSQL real (via pytest fixtures).
- No se permiten mocks de la base de datos; los tests de integracion deben usar datos reales.
- Linting: ruff (reemplaza flake8 + isort + black). Configurado en pyproject.toml.
- Type checking: mypy en modo strict para modulos nuevos.

Migraciones de base de datos

Proceso para cambios en el schema:

1. Generar migracion con: alembic revision --autogenerate -m 'descripcion'
2. Revisar el archivo generado en alembic/versions/ antes de commitear.
3. Nunca modificar una migracion ya aplicada en staging o produccion.
4. Para columnas NOT NULL en tablas con datos existentes: agregar la columna nullable, rellenar datos, luego alterarla a NOT NULL en una segunda migracion.
5. Las migraciones se aplican automaticamente en el deploy via CI/CD.